Domain Specific Language Design (2IMP20): DSL Design and the bigger picture

Loek Cleophas

Ivan Kurtev

Department of Mathematics and Computer Science — Software Engineering and Technology cluster

TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

Introduction

- **Software is omnipresent**, hardly any modern device or equipment is without software
- **Software connects people and devices or equipment** with ever increasing complexity
- **Our society depends on software** and software improves the quality of living, a.o. in the following domains:
  - medical
  - automotive
  - social media

➔ **Software (quality) research is vital for modern society**

Introduction

Software has become leading in high-tech equipment:

- without software no production, no innovation

Increase in the amount of software has raised need for:

- correctness of the software
- efficient software development
- awareness with respect to legacy

Introduction

Introduction

Software evolves, continuous change and growth in:

- size of software (amount of LOC)
- complexity of software
- features in (software) systems
- costs to build software
- tools, libraries, frameworks involved
- number of languages in software systems

➔ **Need to raise level of abstraction**

TNO-ESI AT A GLANCE

SYNOPSIS

- Foundation ESI started in 2002
- ESI acquired by TNO per January 2013
- ~60 staff members many with extensive industrial experience
- 8 part-time professors

FOCUS

Managing complexity of high-tech systems

through
- system architecting
- system reasoning and
- model-driven engineering

delivering
- methodologies validated in cutting-edge industrial practice

PARTNER BOARD

ASML, Canon Canon Production Printing, itec, Philips, Radboud University Nijmegen, Thales, TNO, TU Delft, TU/e, University of Twente, University of Amsterdam, Vanderlande, Capgemini engineering, Strypes

7

SYSTEM COMPLEXITY IS INCREASING!

- Five technological and market trends drive increasing complexity in high-tech systems:

1. **Additional functionality**
   - Number of interfaces and lines of code are **rapidly increasing**
2. **Mass customization**
   - Increased customization of systems at design time to the point where **each system is unique**
3. **Long life times**
   - Systems operate for decades and need to **continuously evolve** after deployment
4. **Increasing autonomy**
   - Systems acting autonomously with **little or no human interaction**
5. **Systems of systems**
   - Interconnected systems of which **nobody is in complete control**

5

ABC OF COMPLEXITY MANAGEMENT

A. **Abstraction**: Identify high-level concepts that hide low-level details
   - Software is programmed in high-level programming languages and not using machine code

B. **Boundedness**: Impose acceptable restrictions on the considered problem space
   - Constraints on environment in which system has to function correctly

C. **Composition**: Divide one problem into multiple independent smaller problems
   - System is decomposed into logical functions that are developed individually

Automation is key to coping with complexity
   - Not a fundamental technique to reduce complexity, rather about **productivity**

8

MANAGING COMPLEXITY WITH MODELS

- **Model-based methodologies** are promising for managing complexity
- There is a clear relation to the ABC Complexity Management Techniques
  - Models are **abstractions** of the system
  - System **boundedness** can be expressed through parameter ranges and constraints in models
  - Systems are (de-)composed into models covering different parts or aspects. Model-based development methodologies are the **composition** of these
- The formal nature of models provide a strong link to **automation**
  - Models are used as a **source** for automated **analysis** and **synthesis**

FOSD 2024 keynote, see https://set.win.tue.nl/event/fosd-meeting-2024/

Model driven software engineering

Model driven software development is a software development methodology

which focuses on creating and exploiting domain models:

abstract representations of the knowledge and activities that govern a specific application domain

Model driven software engineering

Modeling in Science vs in Engineering

- Modeling in science: description of existing artifacts, processes, phenomena, etc.
  - Analytical—goal to understand, explain and predict
  - Capturing complexity
- Modeling in engineering: description of artifacts
  - Constructive—to design, create, construct, produce
  - Mastering complexity

Model driven software engineering

Models are common practice when designing (mechatronic) systems

- hardware design
- electronic design
- physical models
- Matlab/Simulink models
- software models

Software has proven to be crucial but at the same time a challenge

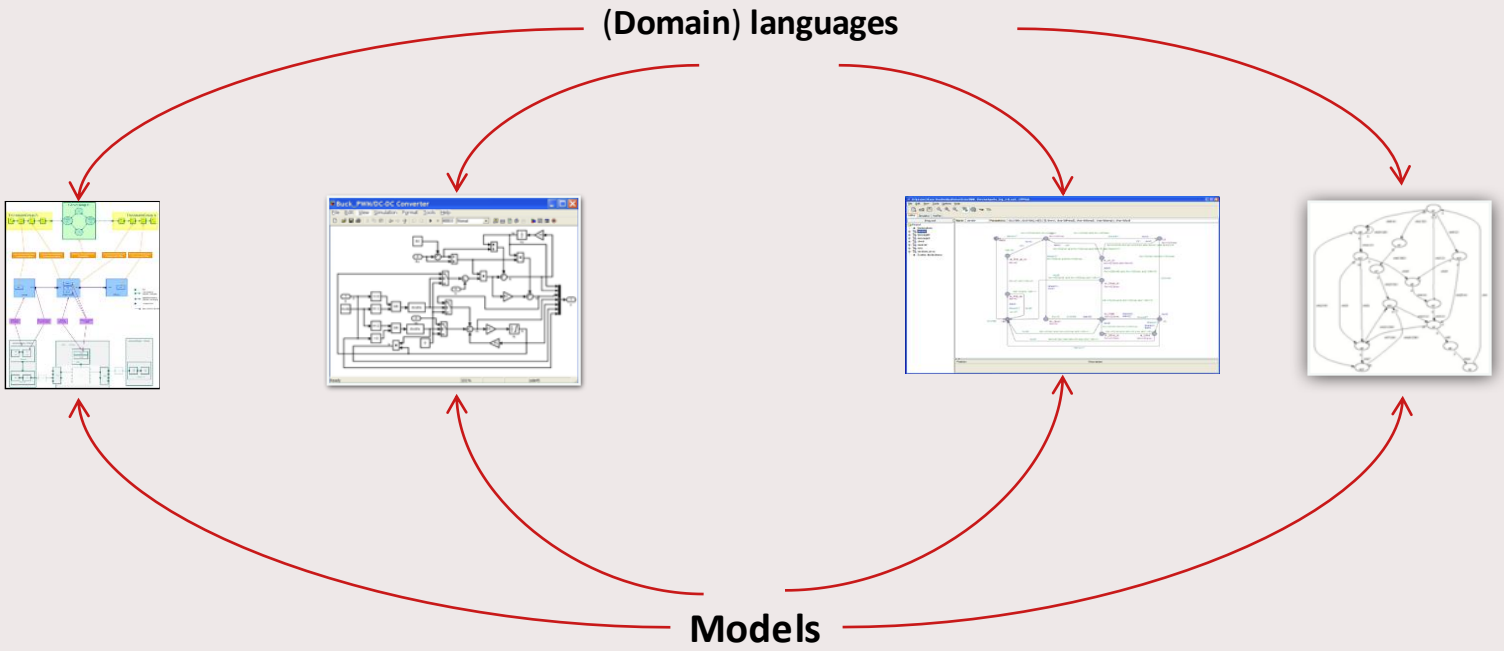
- → model driven engineering has become very popular

Model driven software engineering

Model driven engineering

- considers *models as first-class citizens*
- increases level of abstraction because of the use of models, hiding complexity
- offers the choice between GPLs and DSLs
  - the first may lead to a vendor lock-in
  - the second may involve a huge investment in language design, implementation, and tooling

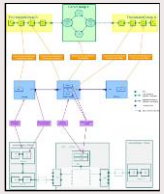
Model driven software engineering

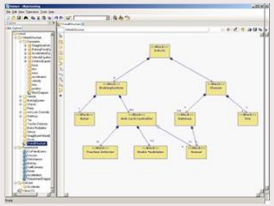


Model driven software engineering

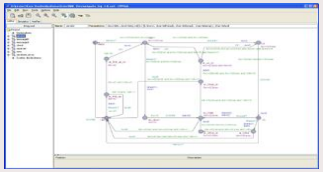
Domain models (specification)

Aspect models (analysis)





Transformations



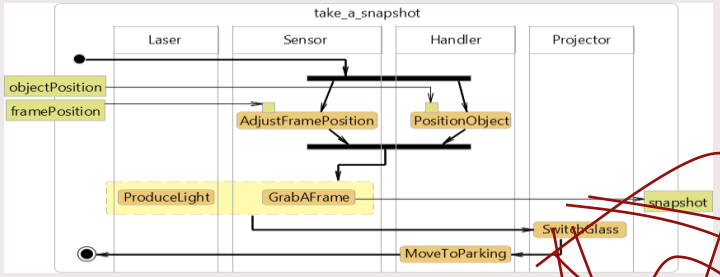
TU/e

Model driven software engineering

- A DSL is a formal, processable language targeting a specific aspect of a system
- Its semantics, flexibility and notation are designed to support working with that aspect as efficiently as possible

- *"A language that offers, through appropriate notations and abstractions, expressive power focused on, and usually restricted to, a particular problem domain"*

Model driven software engineering



Model transformations and Code generation

Model driven software engineering

Domain specific languages offer

- reduction of development time via increase of abstraction and shifting complexity to underlying libraries or model transformations
- increase of robustness via verification of models and model transformations

However, this needs efficient design and implementation of DSLs and corresponding tooling
… and more importantly their maintenance as well

Model driven software engineering

Modeling in practice

- used across disciplines
- abstraction mechanism, used for
  - communication, guidance
  - sketch, blueprint, prototype
  - analysis, verification, optimization
  - automated production

Model driven software engineering

Software systems: more complex than other systems

- Boeing 787:
  - 2.3 million physical parts
- Large software systems:
  - Boeing 787: 14 MLOC
  - Linux: 30MLOC
  - Windows: 50 MLOC (already in 2003!)
  - Modern car: 100+(!) MLOC
- *"Models help us understand (...) potential solutions through abstraction. (...) Software systems, which are often among the most complex engineering systems, can benefit greatly from using models"* (Bran Selic)

Model driven software engineering

- **For what purpose do we create models?**
  - Which criteria do we need to address?
  - What quality aspects do we need to consider?

Model driven software engineering

Modeling

Model of a lightbulb

TU/e

Model driven software engineering

Modeling

Model of a lightbulb

Model criteria

1. *Pragmatic criterium*
2. Reduction criterium
3. Utility criterium

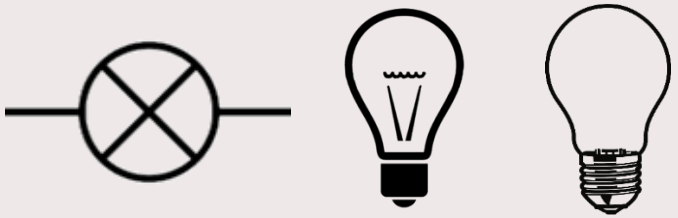
Model driven software engineering

Modeling

Model criteria

1. Pragmatic criterium
2. *Reduction criterium*
3. Utility criterium

Model of a lightbulb



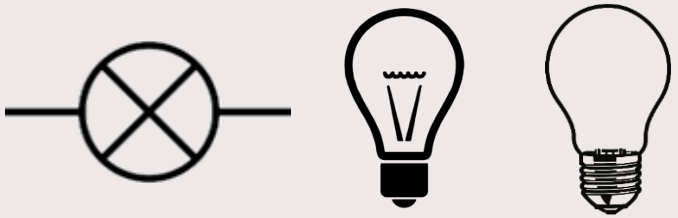
Model driven software engineering

Modeling

Model criteria

1. Pragmatic criterium
2. Reduction criterium
3. *Utility criterium*

Model of a lightbulb



Circuit diagram Store Manufacturer

Model driven software engineering

Modeling

Model of a lightbulb

“All models are wrong, but some are useful”

- George Box (1976)

Model driven software engineering

Modeling

Quality aspects

1. *Understandable*
2. Unambiguous
3. Machine readable

Model driven software engineering

Modeling

Quality aspects

1. Understandable
2. *Unambiguous*
3. Machine readable

Model driven software engineering

Modeling

Quality aspects

1. Understandable
2. Unambiguous
3. *Machine readable*

<lightbulb/>

Domain specific language design

Meta-modeling is modeling modeling languages:

- The Greek prefix “meta” means "about" - meta-model states something "about" other models
- Meta-model describes the *abstract syntax*
- EMF is a popular language for meta-modeling
- Tools can generate editors, serializes, and data structures from meta-models

Domain specific language design

Domain specific language is *“a language that offers, through appropriate notations and abstractions, expressive power focused on, and usually restricted to, a particular problem domain” (van Deursen, et al.)*

1. Domain analysis
2. Language design
3. Language implementation

Domain specific language design

Meta-models interact with requirements elicitation

- AKA: domain analysis

- *Purpose*: What is the purpose of the models? How are they going to be used? What are the intended use cases?

- *Stakeholders*: Who are the key stakeholders and the intended users of the language? Who will create models, and who will read them?

- *Concepts*: What are the key domain concepts that users care about when working on the intended use cases?
- *Concept properties*: What are the relevant properties of the domain concepts?
- *Relations between concepts*: What are the relations between the concepts?

Domain specific language design

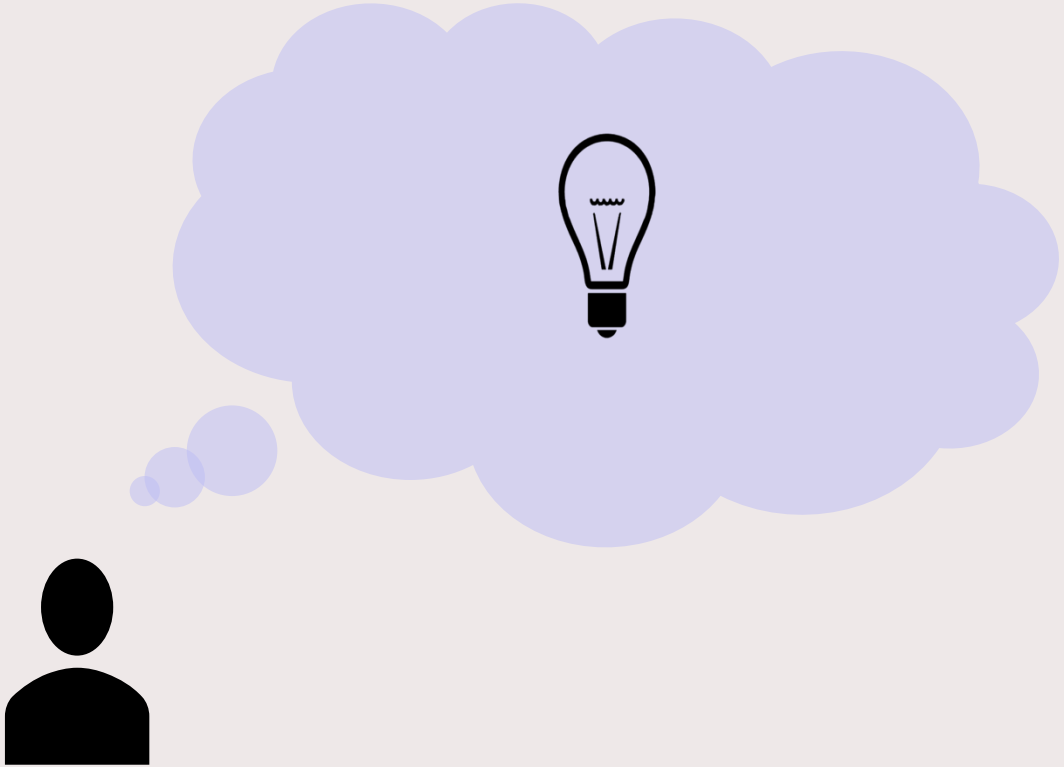
Domain Analysis

- While meta-modeling is clearly not part of usual requirements engineering, dependencies between the domain models and requirements are obvious
- *"A problem domain is defined by consensus, and its essence is the shared understanding of some community"*

Domain specific language design

Domain model

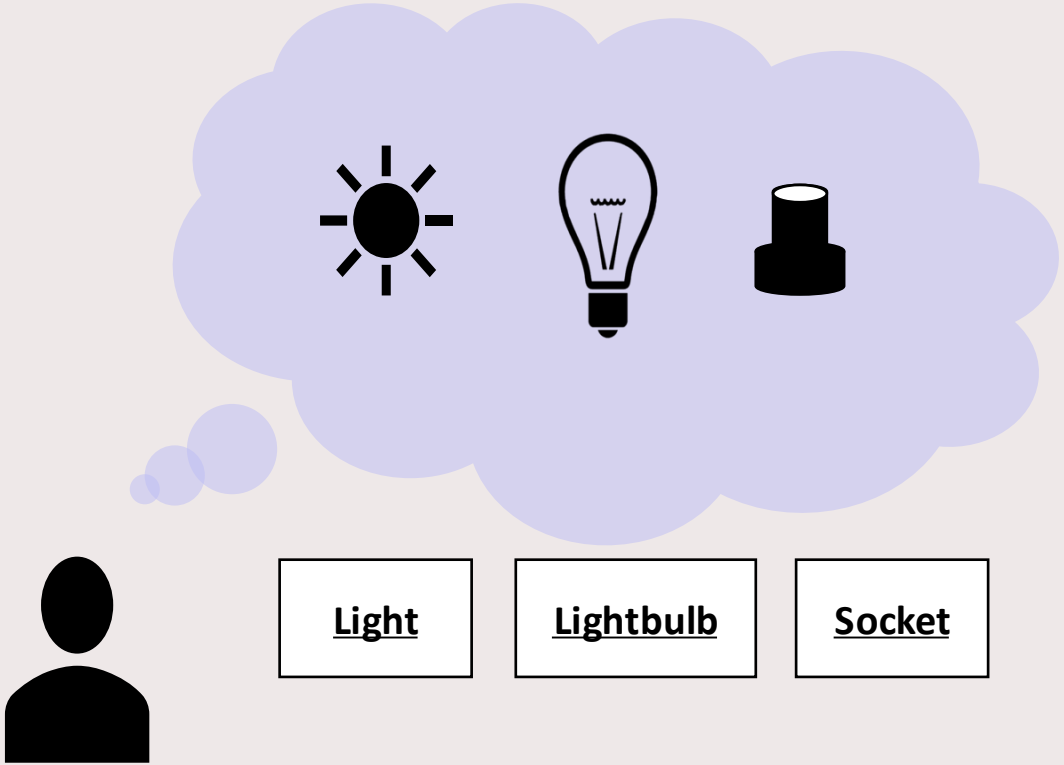
- Conceptual model



Domain specific language design

Domain model

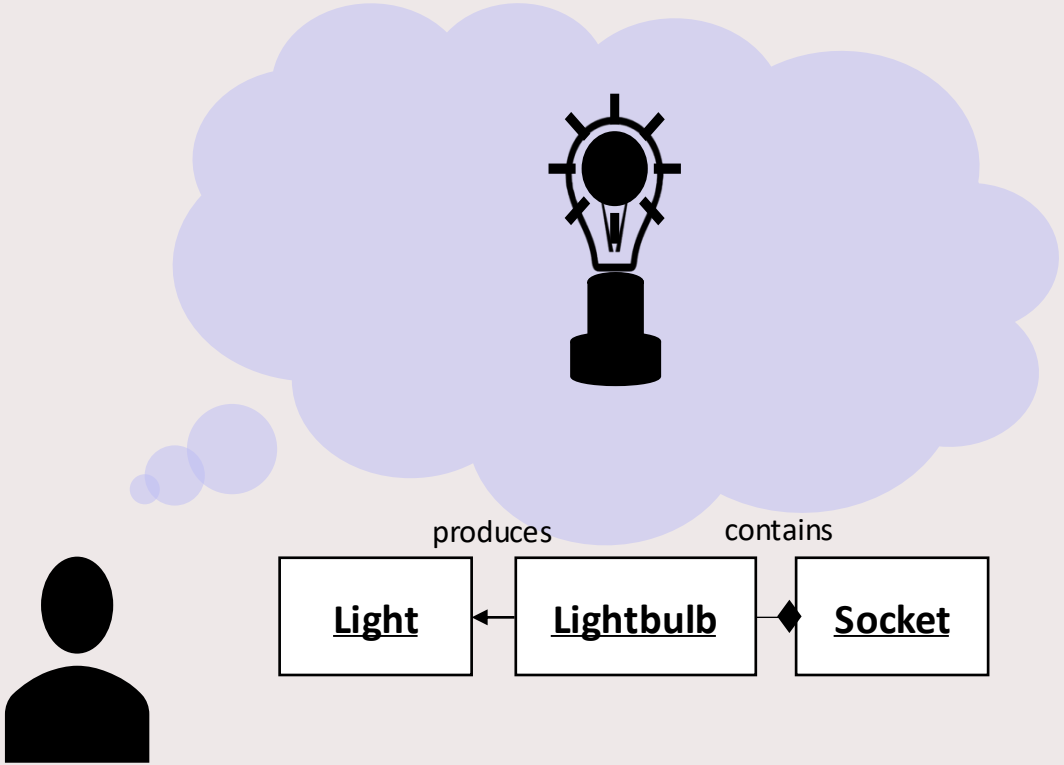
1. *Concepts*
2. Relations
3. Attributes
4. Process



Domain specific language design

Domain model

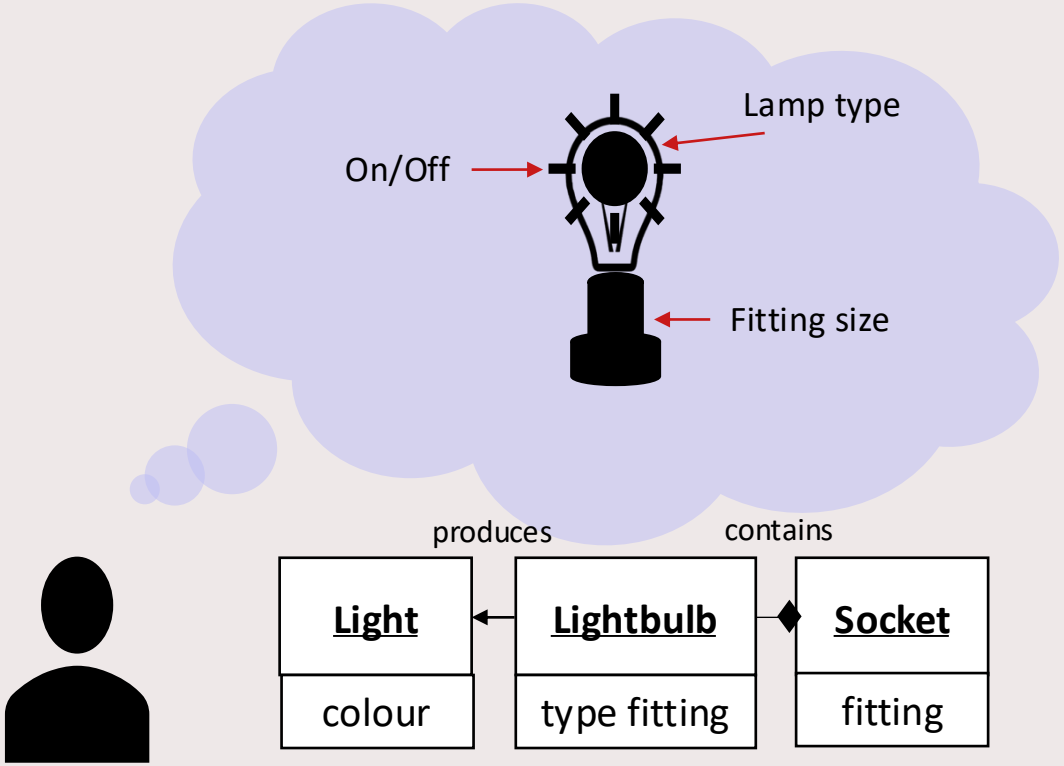
1. Concepts
2. *Relations*
3. Attributes
4. Process



Domain specific language design

Domain model

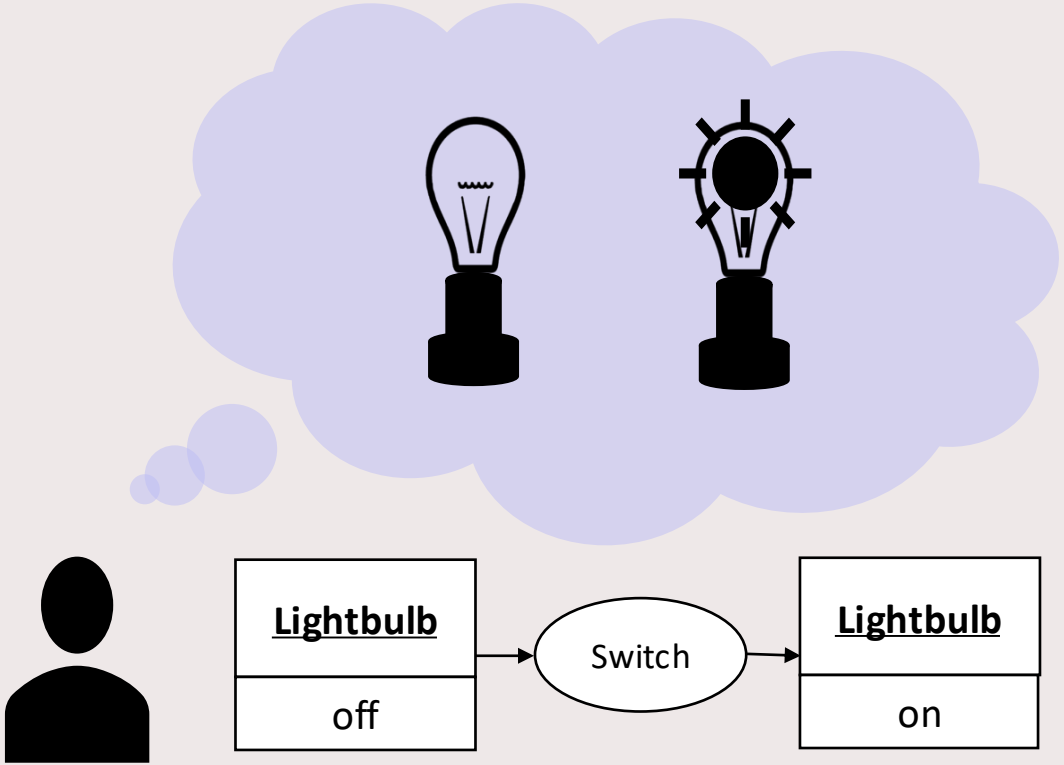
1. Concepts
2. Relations
3. *Attributes*
4. Process



Domain specific language design

Domain model

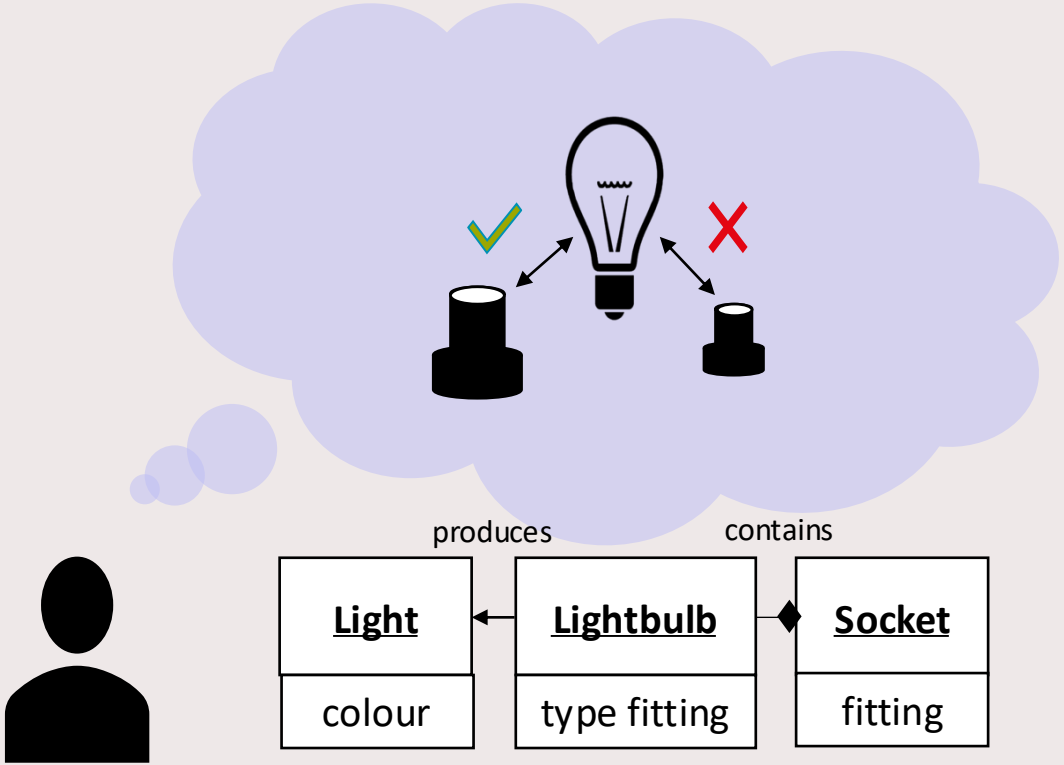
1. Concepts
2. Relations
3. Attributes
4. *Process*



Domain specific language design

Domain model

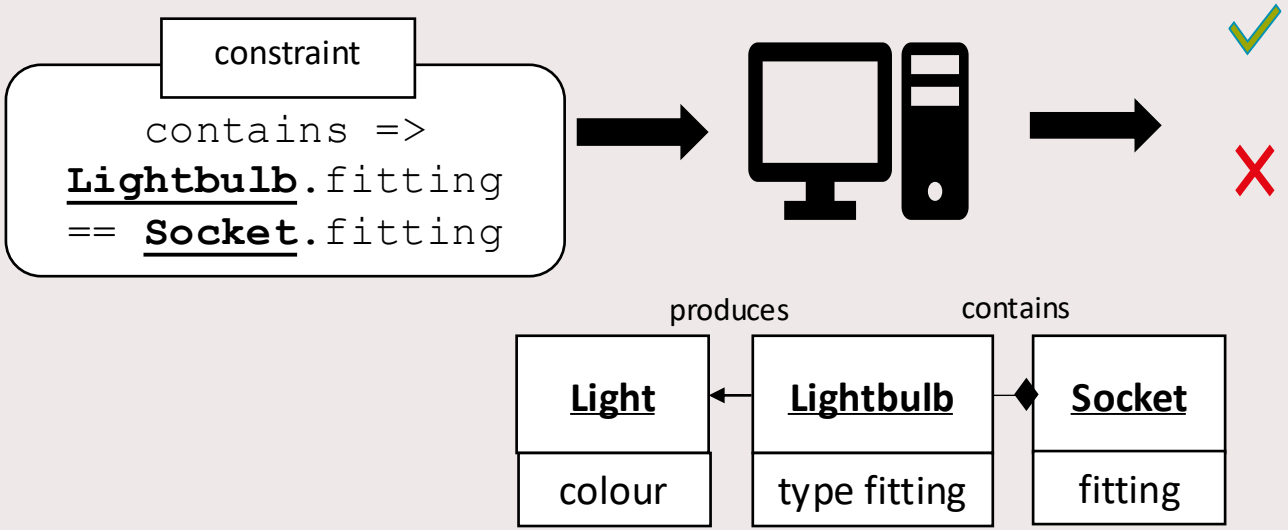
- Model checking



Domain specific language design

Domain model

- Automated model checking



Domain specific language design

Domain model

- Communicatable
- Verifiable

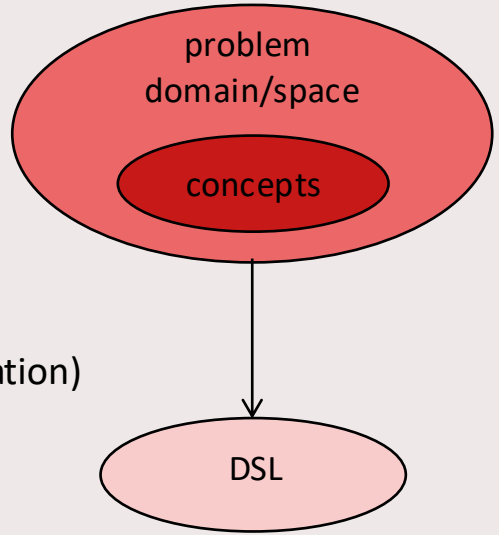
Domain specific language design

Language-driven approach resembles standard software development:

- Design is focused on a DSL
- Stages:
  1. Identification of problem domain/space
  2. Identification of relevant concepts
  3. Formulation of language definition
  4. Implementation (or generation) of language tooling

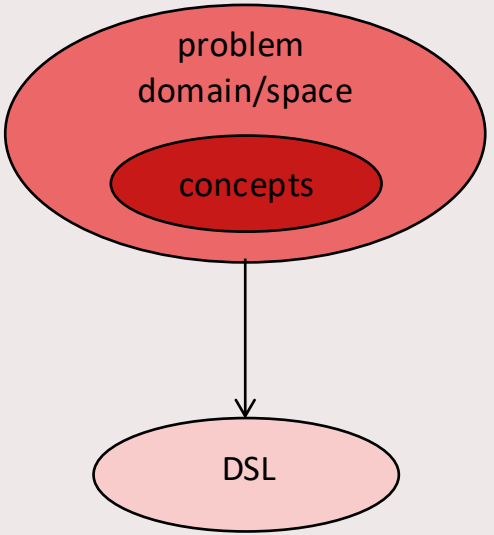
  (1+2 = domain analysis; 3+4 = language design+implementation)

These stages need not be executed sequentially, but may be in parallel



Domain specific language design

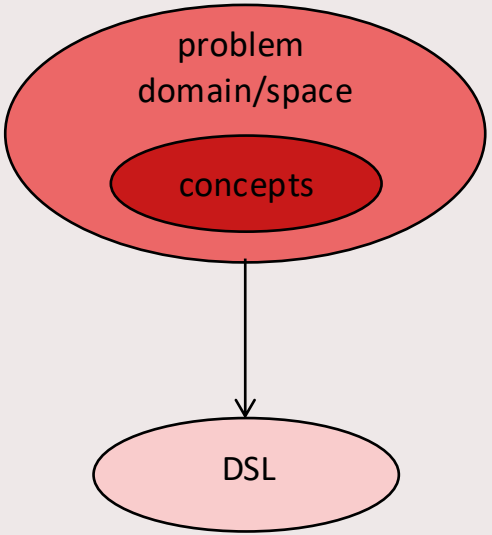
1. Problem domain: reduction of time to introduce new financial products in banks
   - needs to be done more than once
   - capable of capturing in “formal” descriptions
2. Relevant concepts: account, balance, interest, loan, …
   - obtained from interviews, existing software, documentation, guidelines, etc.



TU/e

Domain specific language design

1. Problem domain: reduction of time and cost to test wafer handler module of wafer scanner
   - machines €100+M each
   - need to test bad weather behaviour as well
   - need to test in clean room
   - capable of capturing in “formal” descriptions
2. Relevant concepts: wafer, robot arm, pedestal, sensor, door, …
   - obtained from interviews, existing software, documentation, guidelines, etc.



TU/e

Domain specific language design

Requirements

Specification
In terms of problem domain

Expressive for concise specification of large multi-disciplinary systems

'Look-and-feel' primarily determined by domain experts

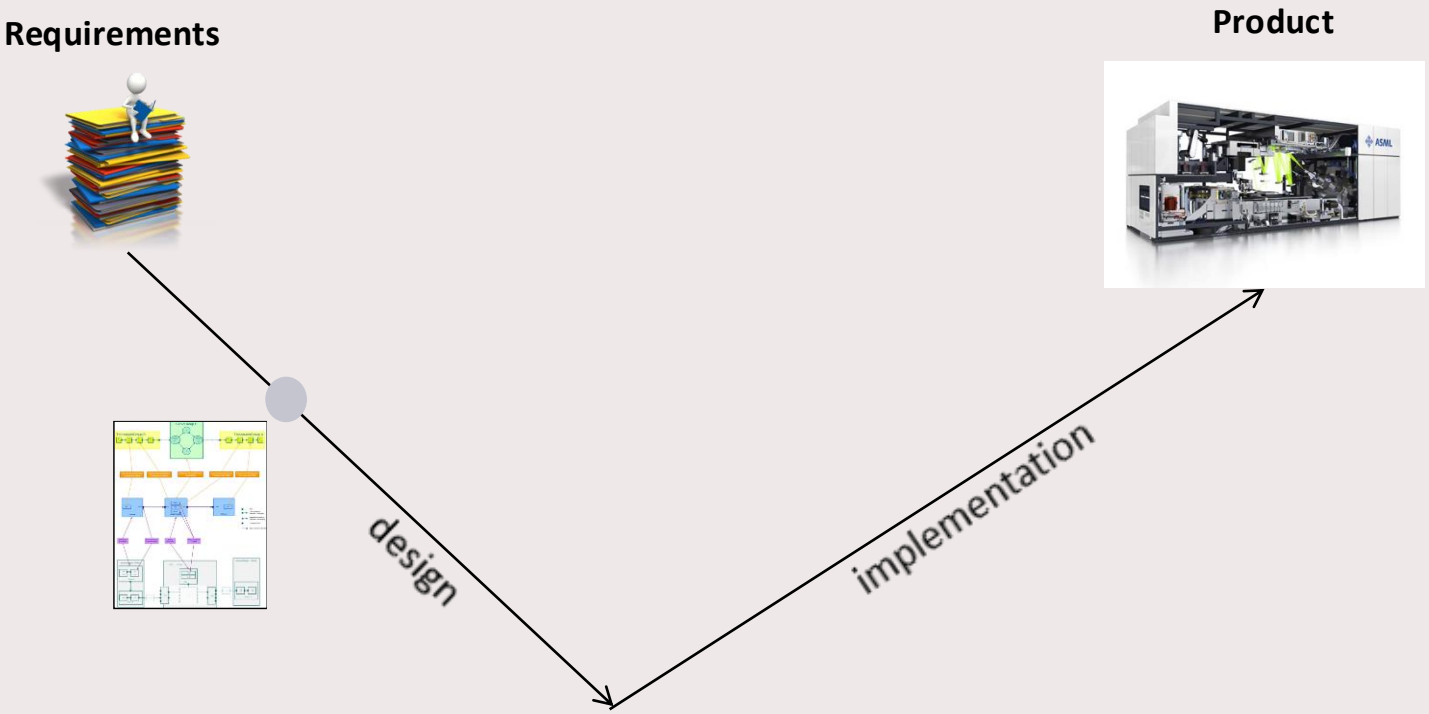
Crucial for adoption

design

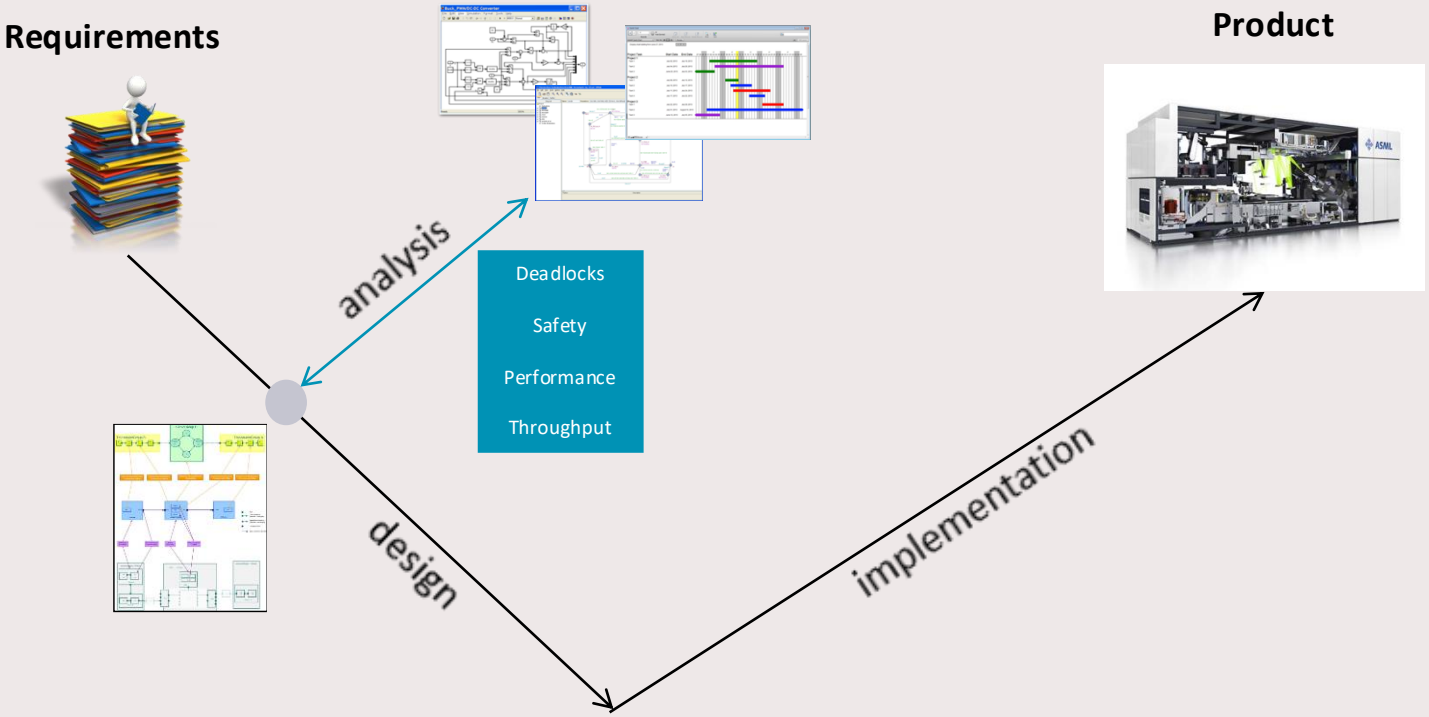
implementation

Product

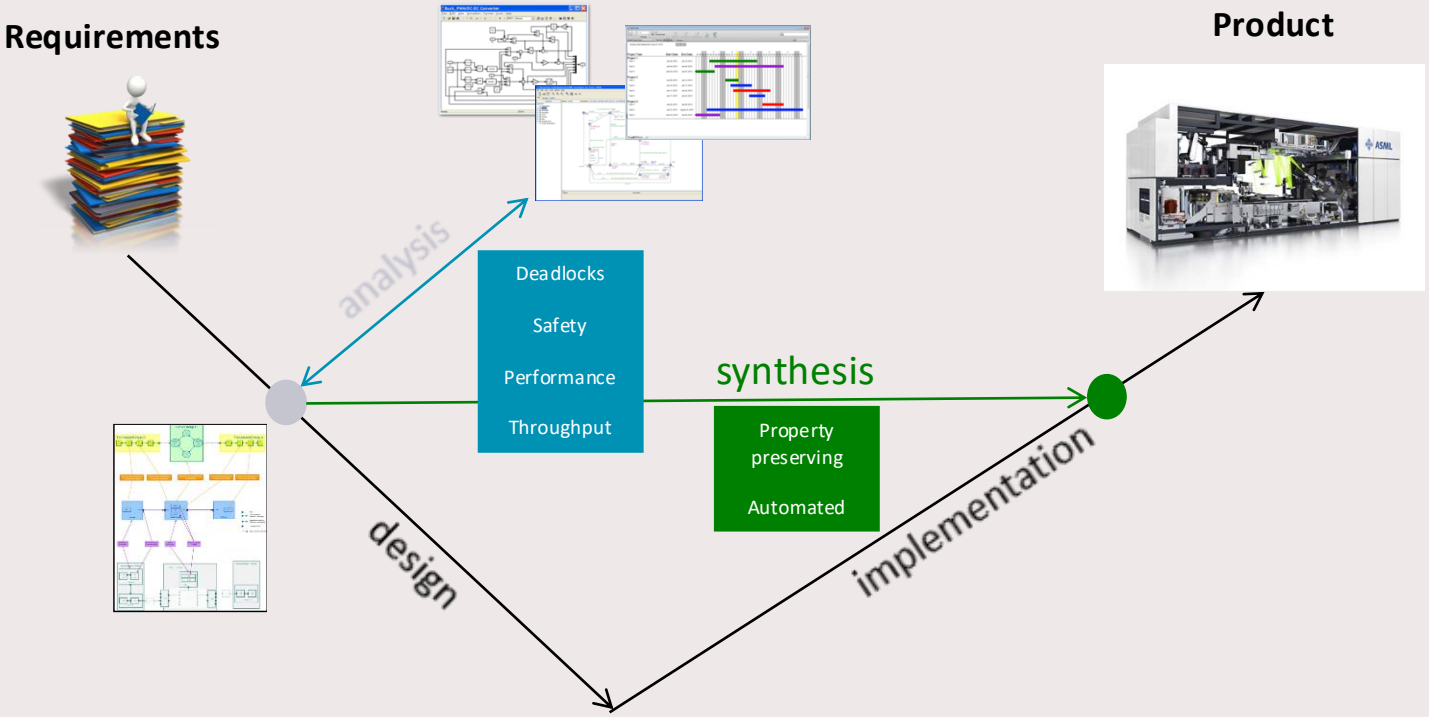
Domain specific language design



Domain specific language design

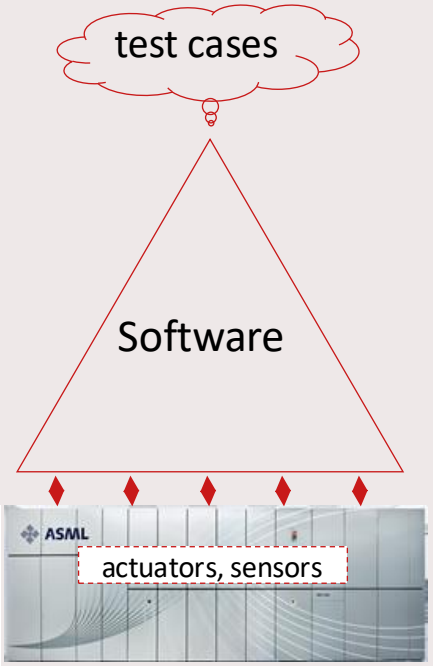


Domain specific language design

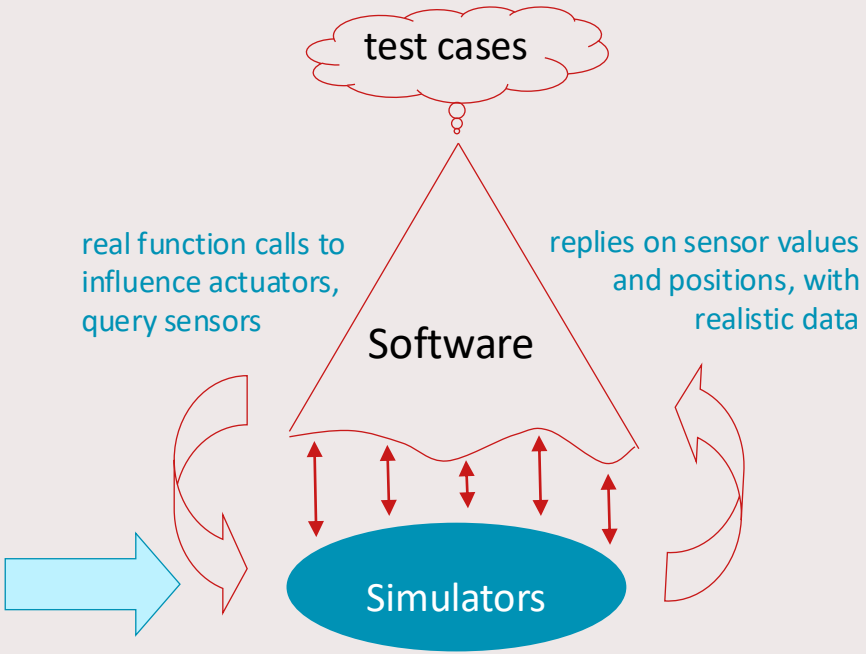


VPDSL

Software in a Loop Simulation

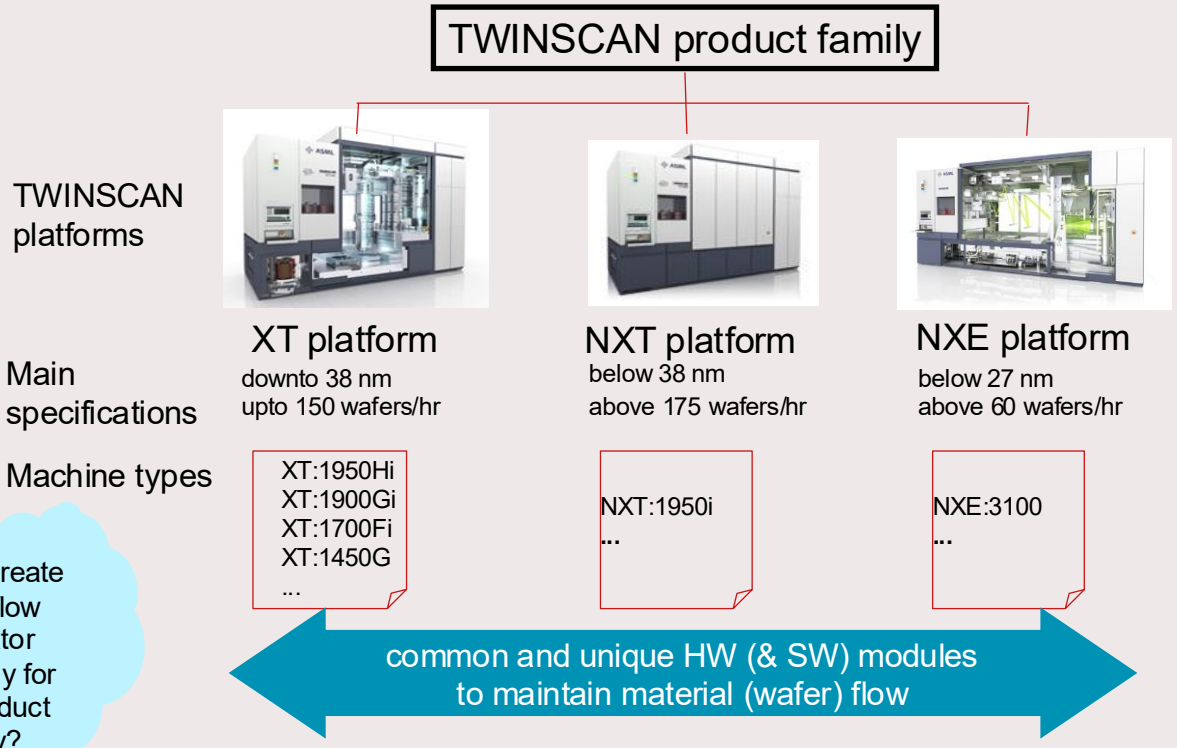


TWINSCAN



VPDSL

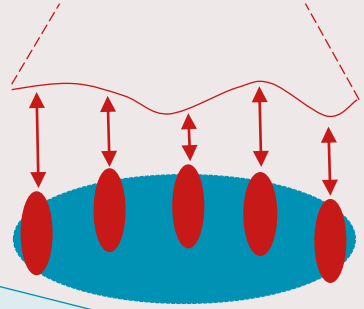
TWINSCAN Platforms and Machine Types



VPDSL

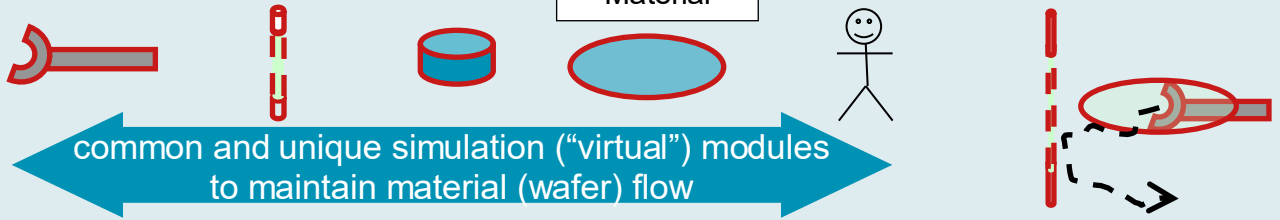
How to Build a Wafer Flow Simulator?

?



…try to re-create a virtual replica of the real world…

Actuators | Sensors | Holders | Transportable Material | Operator | *Interactions* | …

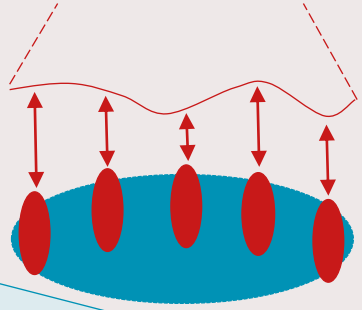


common and unique simulation (“virtual”) modules to maintain material (wafer) flow

VPDSL

How to Build a Wafer Flow Simulator?
Error Scenarios

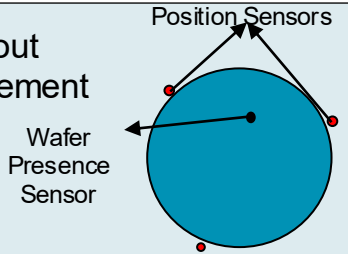
?

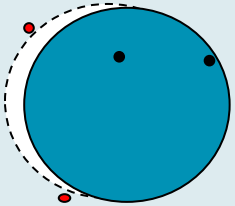


…however, the real world usually does not “behave” perfectly…

Error-injection Scenarios in Simulation

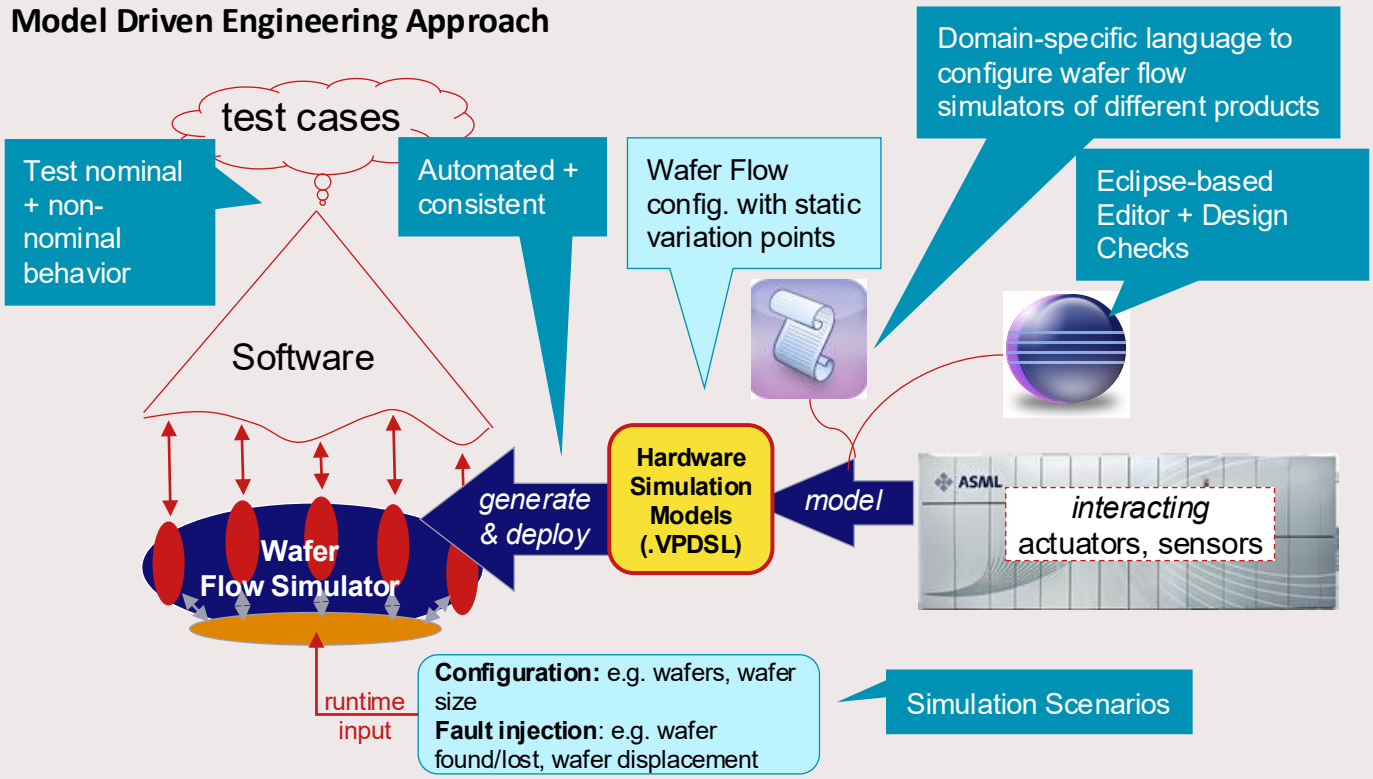
Transfer without wafer displacement





Transfer with wafer displacement

VPDSL


Model Driven Engineering Approach
test cases
Test nominal + non-nominal behavior
Automated + consistent
Wafer Flow config. with static variation points
Domain-specific language to configure wafer flow simulators of different products
Eclipse-based Editor + Design Checks
Software
Wafer Flow Simulator
generate & deploy
Hardware Simulation Models (.VPDSL)
model
ASML
interacting actuators, sensors
runtime input
Configuration: e.g. wafers, wafer size
Fault injection: e.g. wafer found/lost, wafer displacement
Simulation Scenarios

VPDSL

VPDSL – A DSL for Specifying and Generating Wafer Flow Simulators

```
system example_system

peripheral LoadLock-Out
    fixed Pedestal
        transfer_area
            …
        end
    end
    fixed Chamber
        sensor LoadLockSensor-Position
            type lightbeam
            …
            values
                true
                false
                initial false
            end
        end
        …
    end
end

peripheral LoadLock-In …
```

```
actuated LoadRobot
  type 3DOF
  coordinate_system
      origin (0.5, 2.0, -0.1)
  end
  positions
      type polar
      initial (0.2, 0.0, 0.2)
  end
  // implicit transfer area
end

actuated UnloadRobot …

actuated InVacuumRobot …

logical_layout
  LoadRobot -> LoadLock-In
  InVacuumRobot -> LoadLock-In
  InVacuumRobot -> LoadLock-Out
  UnloadRobot -> LoadLock-Out
end
```

Closely resembles the final system – unlike C/C++

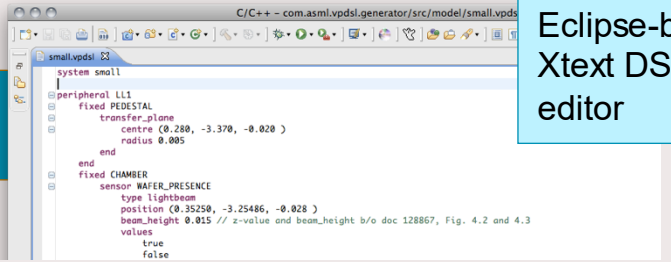
VPDSL

Design Workflow

mechanical engineer

Specify VPDSL model

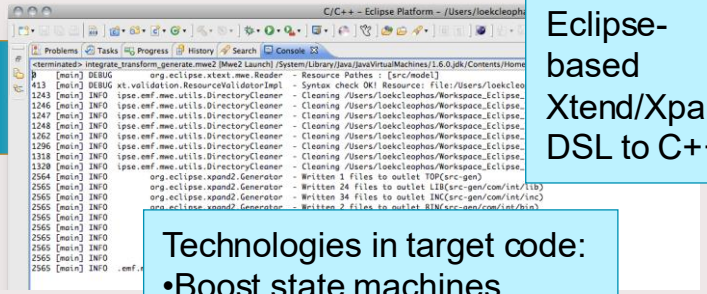
X LoC



Eclipse-based Xtext DSL editor

Generate simulation code

12*X LoC
(8*X model-dependent)



Eclipse-based Xtend/Xpand DSL to C++

Technologies in target code:
- Boost state machines
- Geometry library
- ASML sw. facilities

software engineer

Integrate simulation code with control software

Domain specific language design

Sources of information for the domain analysis can be

- existing libraries/source code
- existing documentation
- stakeholders: end users, domain experts

Domain specific language design

Language prototyping:

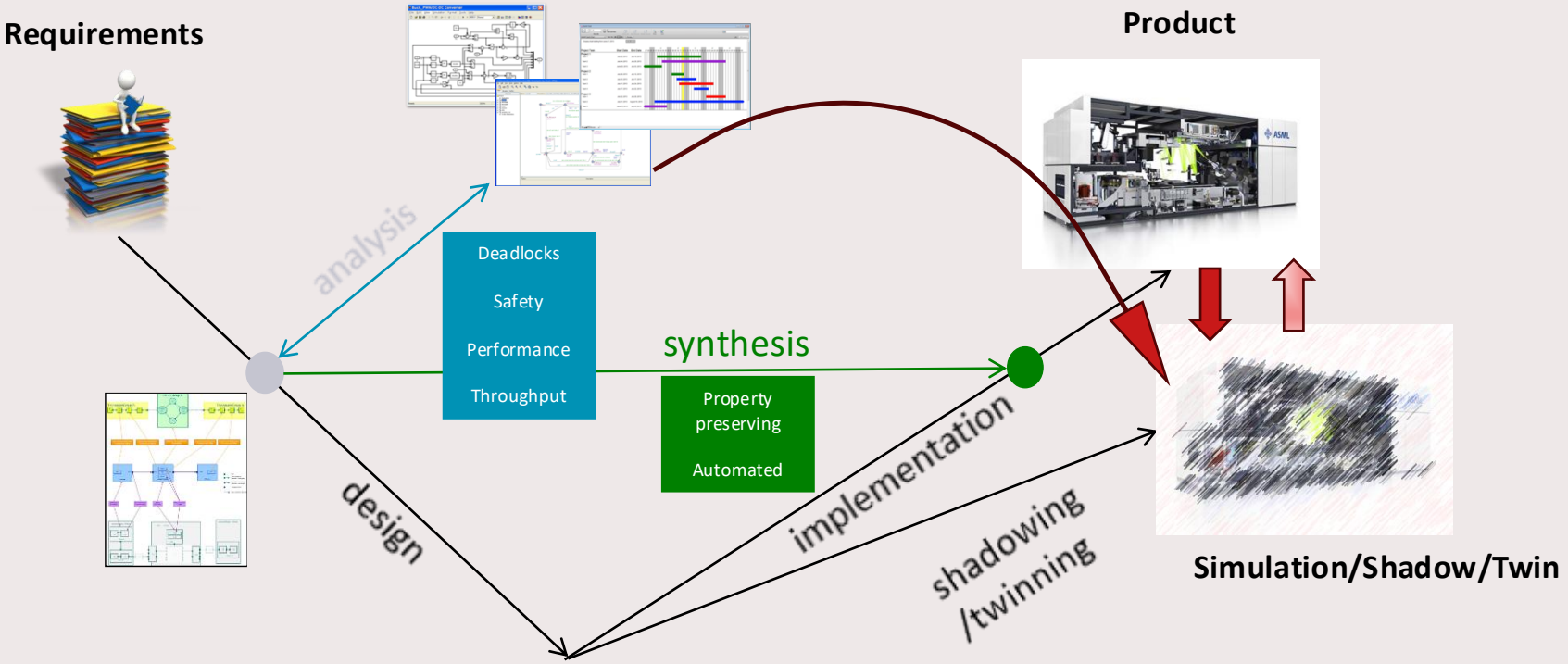
- process
  - separate language engineering from domain engineering
  - iterate over:
    - designing
    - prototyping
    - validating
    - *use examples to explain to stakeholders!*
- tooling
  - use powerful language workbenches
- choose a right release moment

Domain specific language design

A few (research/engineering) challenges:

- Identification of common semantic concepts in a certain domain:
  - High-Tech Industry: real-time, state machines, supervisory control, material flow (paper, wafers), etc.
  - Capturing these concepts in domain specific languages
- Quality and correctness of model transformations/code generators…
  - w.r.t. underlying semantics
- Modularity of meta-models and composition of semantic building blocks
- Evolution of meta-models and co-evolution of models
- Mixing multiple domain specific languages
- Dual-use: for both MDSE of system software, and for MDSE of *digital shadow/twin*

Digital Twinning in an MDSE & DSL context



Lookahead

- 2026/06/10: guest lecture Eugen Schindler, Canon Production printing
- 2026/06/12: guest lecture Mathijs Schuts, TNO-ESI
- 2026/06/17 and 2025/06/19: backup slots

- **Remember**: attending *at least* 2 out of 3 guest lectures is required

- Submit suggested choice of domain + papers by June 10th 23:59
- Get feedback on domain + paper choice latest June 17th
- Submit your work for assignment 3 by Friday July 3rd 23:59